V. Sarcar, *Java Design Patterns*
https://doi.org/10.1007/978-1-4842-7971-7_27

27. MVC Pattern

Vaskaran Sarcar[1]
(1) Garia, Kolkata, India

This chapter covers the MVC pattern.

Definition

Trygve Mikkjel Heyerdahl Reenskaug, a Norwegian computer scientist, introduced the concept of the model–view–controller (MVC) pattern for graphical user interface (GUI) software design. He formulated this during his visit to the Xerox Palo Alto Research Center (PARC) in 1979. The Wikipedia link at https://en.wikipedia.org/wiki/Trygve_Reenskaug includes his quote:

> *"MVC was conceived as a general solution to the problem of users controlling a large and complex data set. The hardest part was to hit upon good names for the different architectural components. Model-View-Editor was the first set. After long discussions, particularly with* **Adele Goldberg**, *we ended with the terms Model-View-Controller."*

Throughout this book, I started with the definitions of the respective patterns. For this pattern, here is Wikipedia's simple definition (see https://en.wikipedia.org/wiki/Model-view-controller):

> *"Model–view–controller (MVC) is a software design pattern commonly used for developing user interfaces that divide the related program logic into three interconnected elements. This is done to separate internal representations of information from the ways information is presented to and accepted from the user."*

My favorite description of MVC comes from Connelly Barnes (http://wiki.c2.com/?ModelViewController). He says,

> *"An easy way to understand MVC: the model is the data, the view is the window on the screen, and the controller is the glue between*

the two.”

Note that instead of calling it the MVC Pattern, some developers prefer to call it MVC Architecture.

Concept

Using this pattern, you separate the user interface logic from the business logic and decouple the major components in such a way that they can be reused efficiently. This approach promotes parallel development.

From the definition, it is apparent that the pattern consists of these major components: model, view, and controller. The controller is placed between the view and model in such a way that they can communicate to each other only through the controller. This model separates the mechanism of how the data is displayed from the mechanism of how the data will be manipulated. Figure 27-1 shows a typical MVC pattern.

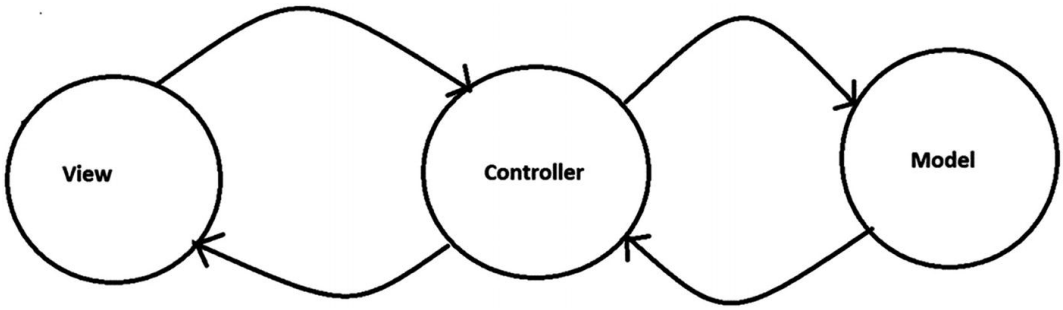

Figure 27-1 A typical MVC architecture

Key Points to Remember

Here are brief descriptions of the key components in this pattern:

- The view layer is used to represent the final output. It can also be used to accept user input. It is a presentation layer, and you can think of it as a GUI. You can design it with various technologies. For example, in a .NET application, you can use HTML, CSS, WPF, etc., and for a Java application, you can use AWT, Swing, JSF, JavaFX, etc. (You may know that AWT is platform-dependent and an older technology. Swing comes after it and it is suitable for desktop applications. JavaFX is newer than Swing and AWT, but it is separate from JDK 11. JavaFX can work in both desktop and mobile applications.)
- The model acts as the actual brain of your application. It manages the data and business logic. It knows how to store, manage, or manipulate

the data and handle the requests that come from the controller. But this component is separate from the view component. A typical example is a database, a file system, or a similar kind of storage. It can be designed with Oracle, SQL Server, DB2, Hadoop, MySQL, and so on.

- The controller is the intermediary. It accepts a user's input from the view component and passes the request to the model. When it gets a response from the model, it passes the data to the view. It can be designed with C#, .NET, ASP.NET, VB.NET, Core Java, JSP, Servlets, PHP, Ruby, Python, and so on.

You may notice varying implementations in different applications. Here are some examples:

- You can have multiple views.
- Views can pass runtime values (for example, using JavaScript) to controllers.
- Your controller can validate the user's input.
- Your controller can receive input in various ways. For example, it can get input from a web request via a URL or input can be passed by clicking a Submit button on a form.
- In some applications, you may also notice that the model can update the view component.

Figures 27-2, 27-3, and 27-4 show known variations of the MVC architecture.

Variation 1

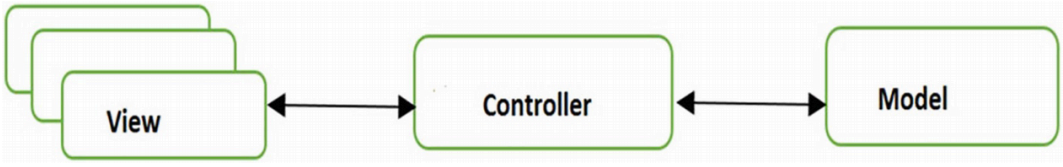

Figure 27-2 An MVC framework with multiple views

Variation 2

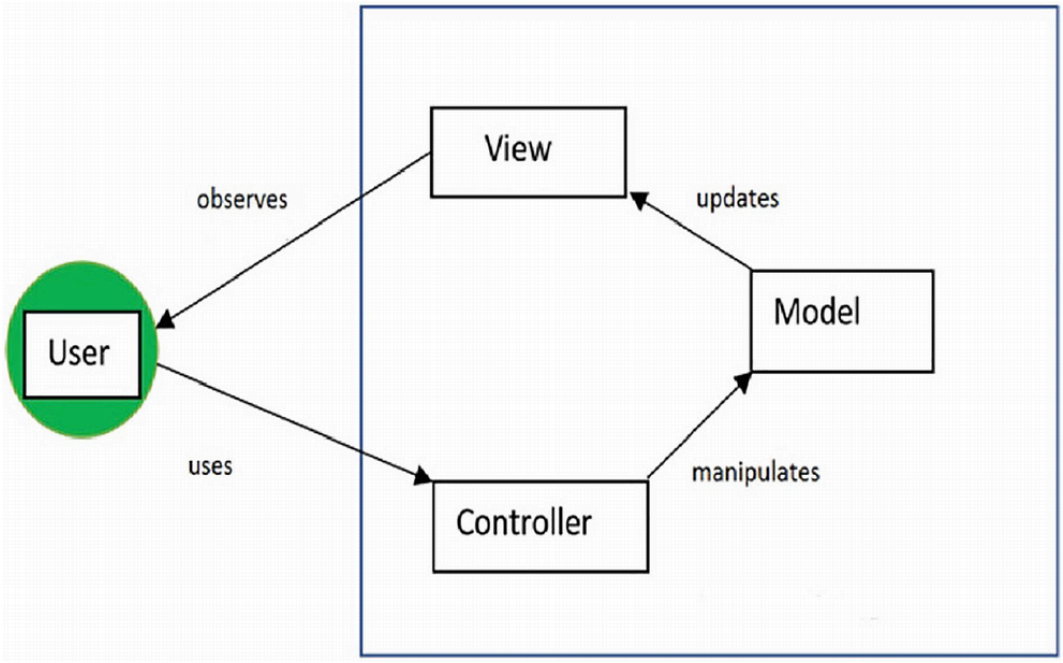

Figure 27-3 A typical MVC framework with a variation

Variation 3

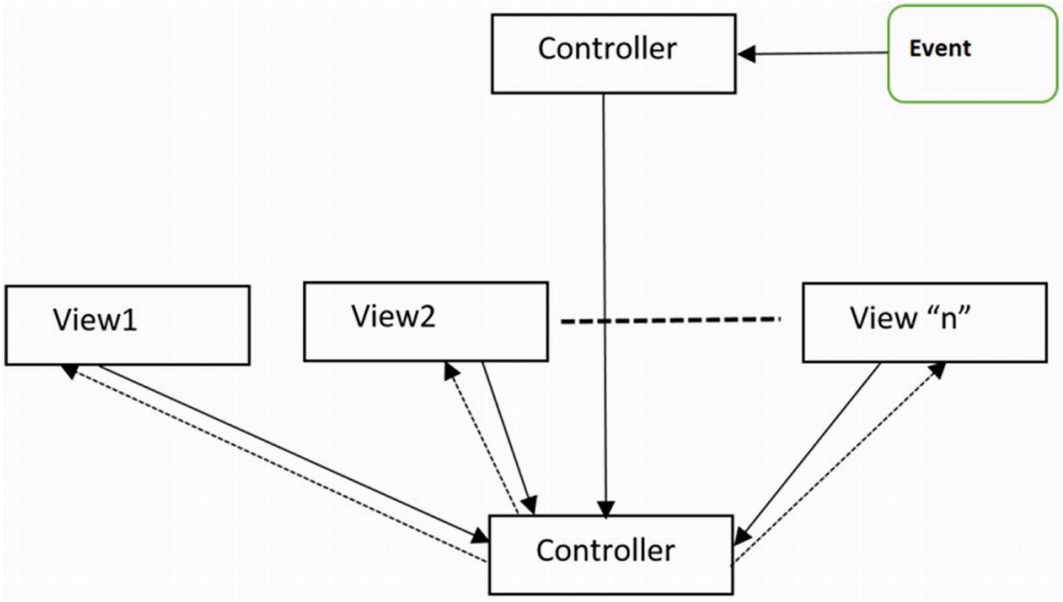

Figure 27-4 An MVC pattern implemented with an Observer pattern/event-based mechanism

In short, you use this pattern to support your own needs.

Real-Life Example

Consider the Template Method pattern's real-life example. But this time let's interpret it differently. I said that in a restaurant, based on customer input, a chef will vary the taste and make final products. But you know that the customer does not place their order directly with the chef. The

customer looks at a menu card and may consult with the server and then place their order. The server passes the order slip to the chef, who gathers the required materials from the restaurant's kitchen (similar to storehouses or computer databases). Once prepared, the server carries the plate to the customer's table. So, you can consider the role of a server as the controller, the menu card as the view, and chefs with their kitchen as the model (and the food preparation materials as data).

Computer World Example

Many web programming frameworks use the concept of the MVC framework. Typical examples include Django, Ruby on Rails, and ASP.NET. In the Java world, in an MVC architecture, the Java servlets can act as controllers and JavaBeans as models whereas JSPs can be used to create different views. In fact, Java's Swing components follow a variation of MVC architecture. If interested, you can go to `www.oracle.com/technical-resources/articles/javase/mvc.html` to understand how to design MVC using Java SE and the Swing libraries.

Since in this section we discuss computer-world examples, here's a non-Java example, too. Consider a typical ASP.NET MVC project's Solution Explorer screenshot, which is shown in Figure 27-5. Notice the pointed arrows in this figure: they represent the views, models, and controllers separately.

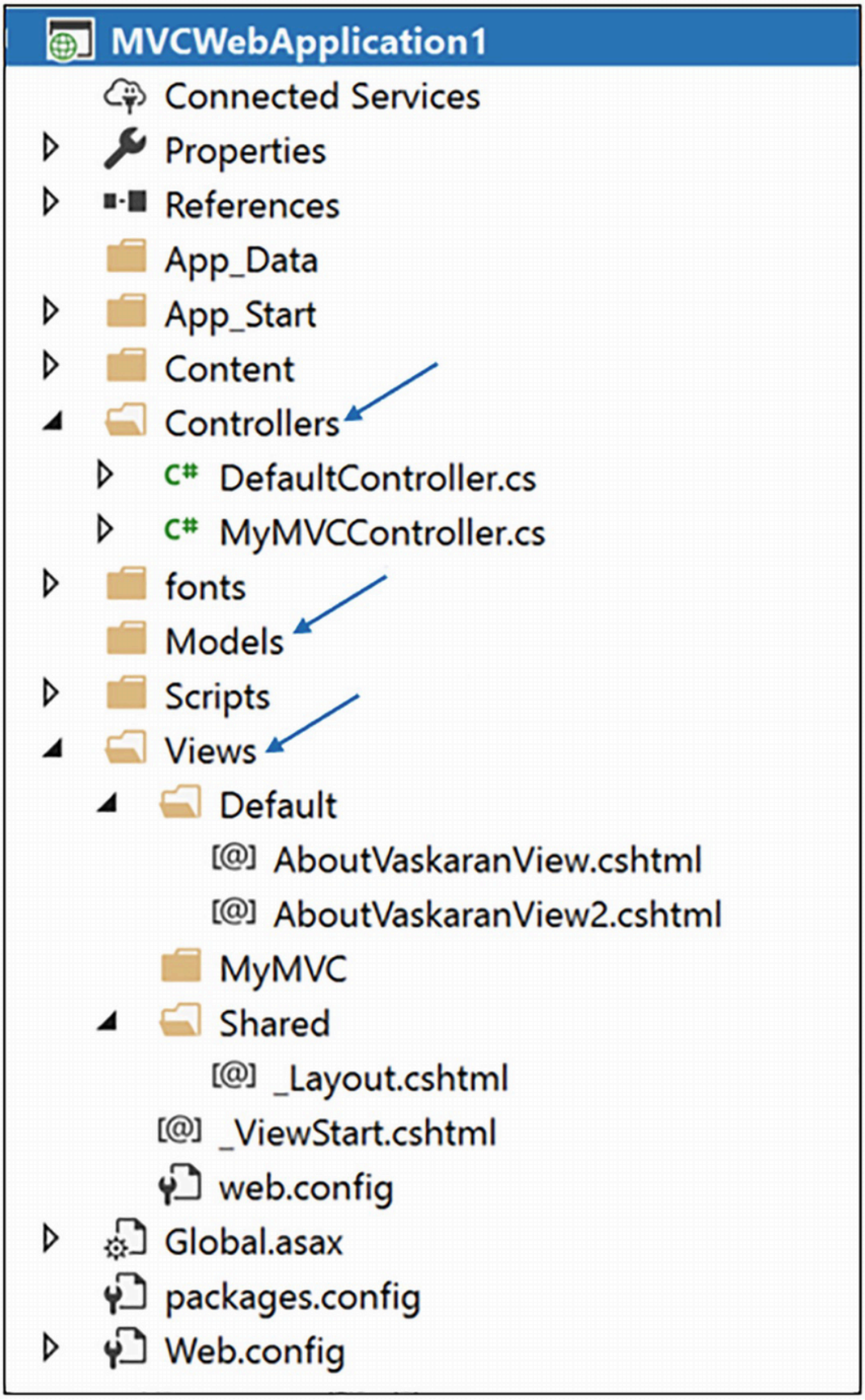

Figure 27-5 Solution Explorer View of a typical ASP.NET MVC project

Points to Note

Different technologies can follow different structures and so you don't need to get a folder structure with the strict naming convention shown in Figure 27-5.

Implementation

One of the best descriptions for MVC comes from wiki.c2.com (http://wiki.c2.com/?ModelViewController) where it says the following: "**We need SMART Models, THIN Controllers, and DUMB Views.**" In the upcoming example, we'll try to implement this suggestion.

Once you examine the Package Explorer view in Figure 27-7, you will understand that the separate folders are created to accomplish the respective tasks. For simplicity and to match the core concept, I also divided the upcoming implementation into three major parts: model, view, and controller. Here are some important points before you read further:

- `Model`, `View`, and `Controller` are three abstract classes that are extended by the concrete classes `EmployeeModel`, `ConsoleView`, and `EmployeeController`, respectively. Seeing these names, you can assume that they are representatives of the Model, View, and Controller layers of the MVC architecture.
- In this application, the requirement is very simple. There are some employees who need to register themselves in an application. Initially the application will register three employees: Amit, Jon, and Sam. Each employee has a name and an ID. Thus the following constructor:

```
public EmployeeModel() {
// Adding 3 employees at the beginning.
    enrolledEmployees = new ArrayList<Employee>();
    enrolledEmployees.add(new Employee("Amit", "E1"));
    enrolledEmployees.add(new Employee("John", "E2"));
    enrolledEmployees.add(new Employee("Sam", "E3"));
}
```

- At any point in time you should be able to see the enrolled employees in the system. In the client code, you can invoke `displayEnrolledEmployees()` on a `Controller` object as follows:

```
controller.displayEnrolledEmployees();
```

- Then the controller retrieves the data from the model and forwards the call to the view layer as follows:

```
// Get the data from the model layer
List<Employee> enrolledEmployees = model.
                              getEnrolledEmployeeDetailsFromModel();
// Connect to the view layer
view.showEnrolledEmployees(enrolledEmployees);
```

- And you'll see that a concrete implementor of the View interface (ConsoleView.java) describes the method as follows:

```
System.out.println("\n ***This is a console view of
                currently enrolled employees.*** ");
 for(Employee employee : enrolledEmployees)
 {
    System.out.println(employee);
 }
System.out.println("----------------------");
```

- You can add a new employee or delete an employee from the registered employees list. To do this, you call `addEmployeee(...)` and `removeEmployee(...)` on a Controller object. These methods, in turn, invoke the `addEmployeeToModel(Employee employee)` and `removeEmployeeFromModel(String employeeIdToRemove)` methods. Seeing the method signature of `removeEmployeeFromModel(...)`, you can guess that to delete an employee, you need to supply the ID of an employee (which is a `String` in this example).
- When a client invokes a delete request, if the employee ID is not found, the application will ignore this delete request.
- Before you add an employee to the application or remove an employee from the application, you perform a simple validation. For example, before you add an employee to an `EmployeeModel` instance, you ensure that you do not add an employee with the same ID repeatedly in the application. So, you see the following code snippet:

```
System.out.println("\nTrying to add an employee with the
                  name: "+ emp.getEmpName()+" and
                  ID: "+emp.getEmpId());
Employee search_emp =
                  findEmployeeWithId(emp.getEmpId());
if (search_emp != null) {
 System.out.print("FAILED! Duplicate ID.");
```

```
    System.out.println(emp.getEmpId() + " is already
                       added to the system.");
  } else {
    enrolledEmployees.add(emp);
    System.out.println(emp + " [is added now.]");
  }
```

The remaining code is easy to understand, and you can go through the complete demonstration now. Yes, it's big but when you analyze it part by part with the help of the previous bullet points and the supporting diagrams, you should not face any difficulties in understanding the code. You can also consider the associated comments for your immediate reference.

Points to Note

Most of the time, you may want to use the concept of MVC with technologies that can give you some built-in support and perform lots of groundwork for you. For example, a .NET programmer often uses ASP.NET (or similar technology) to implement the MVC pattern because it offers a lot of built-in support. Still, you must agree that in these cases, you need to learn the new terminology.

Throughout this book, I use console applications for different design pattern implementations. So, let's continue to use the same for the upcoming implementations because our focus is on MVC architecture only, not on new technologies.

Class Diagram

Figure 27-6 shows the class diagram.

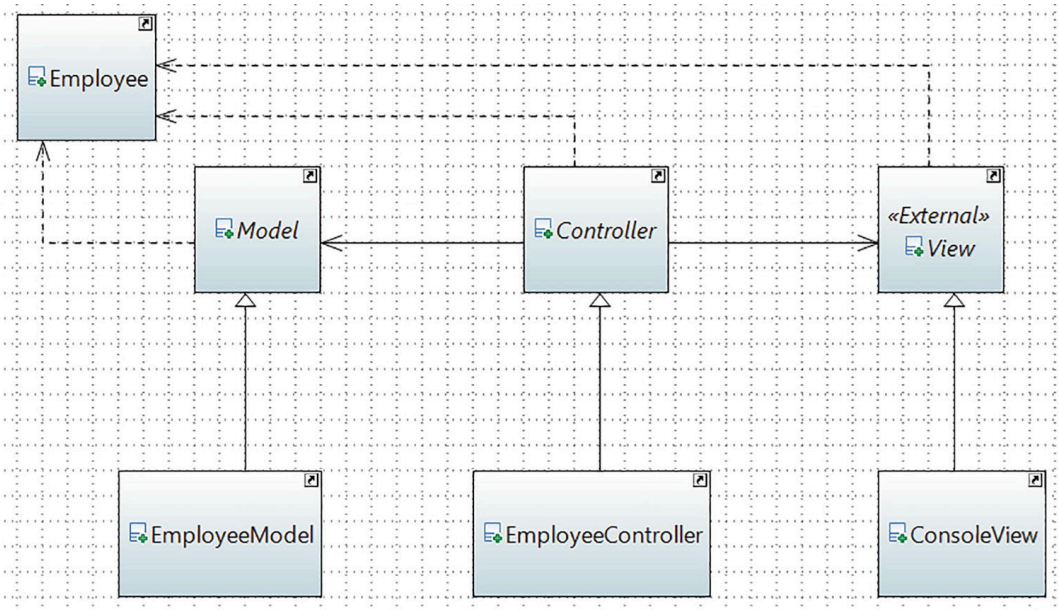

Figure 27-6 Class diagram

Package Explorer View

Figure 27-7 shows the high-level structure of the program. There are many parts to this diagram. To accommodate the size in a single page, I expand some important portions only.

Figure 27-7 Package Explorer view

Demonstration

Here's the complete demonstration.

Contents in Model Folder

// Employee.java

```
package jdp3e.mvc.model;
// The key "data" in this application
public class Employee {
     private String empName;
     private String empId;
     public String getEmpName() {
```

```
            return empName;
      }
      public String getEmpId() {
            return empId;
      }



      public Employee(String empName, String empId) {
            this.empName = empName;
            this.empId = empId;
      }
      @Override
      public String toString() {
            return empName + "'s employee id is: " + empId;
      }
}
```

// Model.java

```
package jdp3e.mvc.model;
import java.util.List;
public abstract class Model {
      public abstract List<Employee>
                  getEnrolledEmployeeDetailsFromModel();
      public abstract void addEmployeeToModel(
                                    Employee employeee);
      public abstract void removeEmployeeFromModel(
                                    String employeeId);
}
```

// EmployeeModel.java

```
package jdp3e.mvc.model;
import java.util.ArrayList



;
import java.util.List;
public class EmployeeModel extends Model {
      private List<Employee> enrolledEmployees;
      public EmployeeModel() {
```

```
        // Adding 3 employees at the beginning.
        enrolledEmployees = new ArrayList<Employee>();
        enrolledEmployees.add(new Employee("Amit", "E1"));
        enrolledEmployees.add(new Employee("John", "E2"));
        enrolledEmployees.add(new Employee("Sam", "E3"));
    }
    @Override
    public List<Employee>
                    getEnrolledEmployeeDetailsFromModel() {
        return enrolledEmployees;
    }
    // Adding an employee to the model
    @Override
    public void addEmployeeToModel(Employee emp) {
        System.out.println("\nTrying to add an employee
                with the name: "+ emp.getEmpName()+" and ID:
                  "+emp.getEmpId());
        Employee search_emp =
                    findEmployeeWithId(emp.getEmpId());
        if (search_emp != null) {



            System.out.print("FAILED! Duplicate ID.");
            System.out.println(emp.getEmpId() + " is
                          already added to the system.");
        } else {
            enrolledEmployees.add(emp);
            System.out.println(emp + " [is added now.]");
        }
    }
    // Removing an employee from model
    @Override
    public void removeEmployeeFromModel(String employeeIdToRemove) {
        System.out.println("\nTrying to remove the employee
                          with id: " + employeeIdToRemove);
        Employee emp =
                    findEmployeeWithId(employeeIdToRemove);
        if (emp != null) {
            System.out.println("Removing this employee.");
            enrolledEmployees.remove(emp);
        } else {
            System.out.println("At present, there is no
                employee with id: " + employeeIdToRemove);
            System.out.println("So, this request is ignored.");
        }
```

```
     }



     Employee findEmployeeWithId(String employeeId) {
          Employee searchEmp = null;
          for (Employee emp : enrolledEmployees) {
               if (emp.getEmpId().equals(employeeId)) {
                    System.out.println(" Employee Found. " +
                              emp.getEmpName() + " has id: " +
                              employeeId);
                    searchEmp = emp;
               }
          }
          return searchEmp;
     }
}
```

Contents in View Folder

// View.java

```
package jdp3e.mvc.view



;
import java.util.List;
import jdp3e.mvc.model.Employee;
public abstract class View {
     public abstract void showEnrolledEmployees(
                         List<Employee> enrolledEmployees);
}
```

// ConsoleView.java

```
package jdp3e.mvc.view;
import java.util.List;
import jdp3e.mvc.model.Employee;
public class ConsoleView extends View {



```

```
     @Override
     public void showEnrolledEmployees(List<Employee>enrolledEmployees
          System.out.println("\n ***This is a console view of
                         currently enrolled employees.*** ");
          for( Employee employee : enrolledEmployees){
               System.out.println(employee);
          }
          System.out.println("----------------------");
     }
}
```

Contents in Controller Folder

// Controller.java

```
package jdp3e.mvc.controller;
import jdp3e.mvc.model.Employee;
import jdp3e.mvc.model.Model




;
import jdp3e.mvc.view.View;
public abstract class  Controller {
     protected Model model;
     protected View view;
     public Controller(Model model, View view){
          this.model = model;
          this.view = view;
     }
     public abstract void displayEnrolledEmployees();
     public abstract void addEmployee(Employee employee);
     public abstract void removeEmployee(String employeeId);
}
```

// EmployeeController.java

```
package jdp3e.mvc.controller;
import java.util.List;
import jdp3e.mvc.model.*;
import jdp3e.mvc.view.*;
public class EmployeeController extends Controller {
```

```
     public EmployeeController(Model model, View view){
          super(model,view);
     }
     @Override
     public void displayEnrolledEmployees() {
          // Get the data from the model layer
          List<Employee> enrolledEmployees =
           model.getEnrolledEmployeeDetailsFromModel();
          // Connect to the view layer
          view.showEnrolledEmployees(enrolledEmployees);
     }
     // Sending a request to model to add an
     // employee to the list



.
     @Override
     public void addEmployee(Employee employee){
          model.addEmployeeToModel(employee);
     }
     // Sending a request to model to remove
     // an employee from the list.
     @Override
     public void removeEmployee(String employeeId){
          model.removeEmployeeFromModel(employeeId);
     }
}
```

Client Code

// Client.java

```
package jdp3e.mvc;
import jdp3e.mvc.model.*;
import jdp3e.mvc.view.*;
import jdp3e.mvc.controller.*;
class Client {



     public static void main(String[] args) {
```

```
        System.out.println("***MVC architecture Demo***");
            // Model
            Model model = new EmployeeModel();
            // View
            View view = new ConsoleView();
            // Controller
            Controller controller = new
                       EmployeeController(model, view);
            controller.displayEnrolledEmployees();
            // Add an employee



            Employee kevin=new Employee("Kevin", "E4");
            controller.addEmployee(kevin);
            controller.displayEnrolledEmployees();
            // Remove an existing employee using
            // the employee id.
            controller.removeEmployee("E2");
            controller.displayEnrolledEmployees();
            // Cannot remove an employee who does not
            // belong to the registered list.
            controller.removeEmployee("E5");
            controller.displayEnrolledEmployees();
            // Avoiding duplicate entries.
            controller.addEmployee(kevin);
            controller.addEmployee(new Employee("Kate", "E4"));
        }
}
```

Output

When you run this program, you see the following output:

```
***MVC architecture Demo***
 ***This is a console view of currently enrolled employees.***
Amit's employee id is: E1



John's employee id is: E2
Sam's employee id is: E3
---------------------
Trying to add an employee with the name: Kevin and ID: E4
```

```
Kevin's employee id is: E4 [is added now.]
 ***This is a console view of currently enrolled employees.***
Amit's employee id is: E1
John's employee id is: E2



Sam's employee id is: E3
Kevin's employee id is: E4
---------------------
Trying to remove the employee with id: E2
 Employee Found. John has id: E2
Removing this employee.
 ***This is a console view of currently enrolled employees.***
Amit's employee id is: E1
Sam's employee id is: E3
Kevin's employee id is: E4
---------------------
Trying to remove the employee with id: E5
At present, there is no employee with id: E5
So, this request is ignored.
 ***This is a console view of currently enrolled employees.***
Amit's employee id is: E1
Sam's employee id is: E3
Kevin's employee id is: E4
---------------------
Trying to add an employee with the name: Kevin and ID: E4
 Employee Found. Kevin has id: E4
FAILED! Duplicate ID.E4 is already added to the system.
Trying to add an employee with the name: Kate and ID: E4
 Employee Found. Kevin has id: E4
FAILED! Duplicate ID.E4 is already added to the system.
```

Q&A Session

27.1 Suppose you have a programmer, a DBA, and a graphic designer. Can you predict their roles in an MVC architecture?

The graphic designer will design the view layer, the DBA will make the model, and the programmer will work to make an intelligent controller. Obviously, if needed, a programmer can work in multiple places.

27.2 What are the key advantages of using the MVC design pattern?

Some important advantages are as follows:

- High cohesion and low coupling are the benefits of MVC. You have probably noticed that tight coupling between the model and the view is easily removed in this pattern. So, the application can be easily extendable and reusable.
- The pattern supports parallel development.
- You can also accommodate multiple runtime views.

27.3 What are the challenges associated with the MVC pattern?

Here are some challenges:

- It requires highly skilled personnel.
- For a tiny application, it may not be suitable.
- Developers may need to be familiar with multiple languages, platforms, and technologies.
- Multi-artifact consistency is a big concern because you are separating the overall project into three major parts.

27.4 Can you provide multiple views in this implementation?

Sure. Let's add a new shorter view called `MobileDeviceView` in the application. Let's add the following code snippet inside the `View` folder as follows:

```
package jdp3e.mvc.view;
import java.util.List;
import jdp3e.mvc.model.Employee;
public class MobileDeviceView extends View {
     @Override
     public void showEnrolledEmployees(List<Employee>enrolledEmployees
          System.out.println("\n ***This is a mobile view of
                         currently enrolled employees.*** ");
          System.out.println("Employee Id"+ "\t"+ " Employee Name");
          System.out.println("_____________________");
          for( Employee employee : enrolledEmployees){
               System.out.println(employee.getEmpId() + "\t"+
                                  employee.getEmpName());
          }
          System.out.println("++++++++++++++++++++");
     }
}
```

Once you add this class, your modified Package Explorer view should be similar to Figure 27-8.

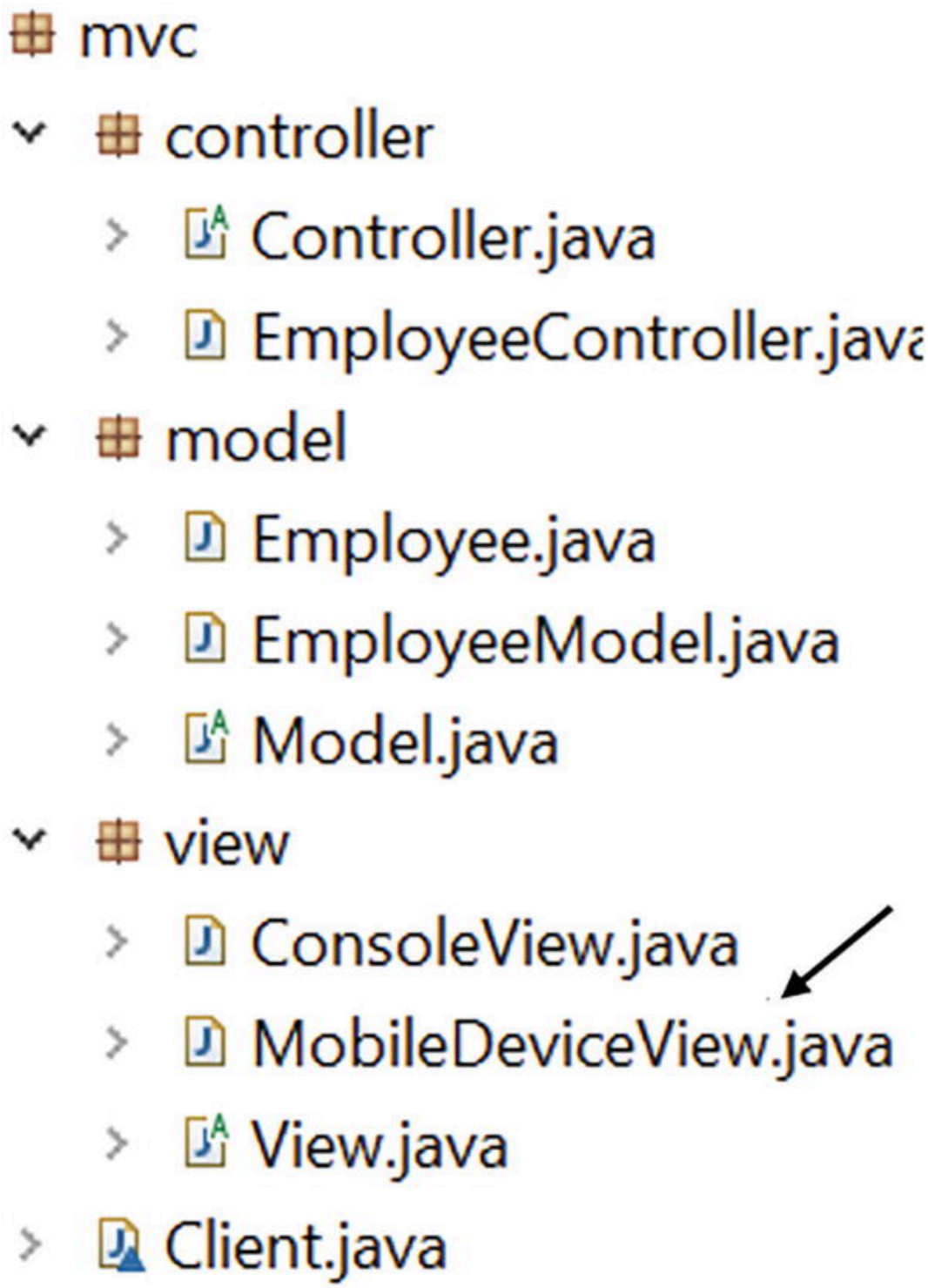

Figure 27-8 New Package Explorer view after the addition of MobileDeviceView.java

Now add the following segment of code at the end of your client code (see the comment for reference):

```
// Added for Q&A Session
view = new MobileDeviceView();
controller = new EmployeeController(model, view);
```

```
controller.displayEnrolledEmployees();
```

Modified Output

Notice the last part of this new output to see the effect of your new changes. These changes are shown in bold.

```
***MVC architecture Demo***
 ***This is a console view of currently enrolled employees.***
Amit's employee id is: E1
John's employee id is: E2
Sam's employee id is: E3


---------------------
Trying to add an employee with the name: Kevin and ID: E4
Kevin's employee id is: E4 [is added now.]
 ***This is a console view of currently enrolled employees.***
Amit's employee id is: E1
John's employee id is: E2
Sam's employee id is: E3
Kevin's employee id is: E4
---------------------
Trying to remove the employee with id: E2
 Employee Found. John has id: E2
Removing this employee.
 ***This is a console view of currently enrolled employees.***
Amit's employee id is: E1
Sam's employee id is: E3
Kevin's employee id is: E4
---------------------
Trying to remove the employee with id: E5
At present, there is no employee with id: E5
So, this request is ignored.
 ***This is a console view of currently enrolled employees.***
Amit's employee id is: E1
Sam's employee id is: E3
Kevin's employee id is: E4
---------------------
Trying to add an employee with the name: Kevin and ID: E4
 Employee Found. Kevin has id: E4
FAILED! Duplicate ID.E4 is already added to the system.
Trying to add an employee with the name: Kate and ID: E4
 Employee Found. Kevin has id: E4
FAILED! Duplicate ID.E4 is already added to the system.
```

```
***This is a mobile view of currently enrolled employees.***
Employee Id      Employee Name



_________________________
E1      Amit
E3      Sam
E4      Kevin
++++++++++++++++++++
```

Analysis

This modified example shows how to use a new view. Similarly, you can add a new model too. For example, you can add a new model named `SqlDataModel` as follows:

```
package jdp3e.mvc.model


;
import java.util.ArrayList;
import java.util.List;
public class SqlDataModel extends Model {
     List<Employee> enrolledEmployees;
     public SqlDataModel() {
     // Adding 4 employees at the beginning.
     enrolledEmployees = new ArrayList<Employee>();
     enrolledEmployees.add(new Employee("Amit", "E1"));
     enrolledEmployees.add(new Employee("John", "E2"));
     enrolledEmployees.add(new Employee("Sam", "E3"));
     enrolledEmployees.add(new Employee("Patrick", "E4"));
 }
// Remaining code skipped
```

In this case, the Package Explorer view is shown in Figure 27-9.

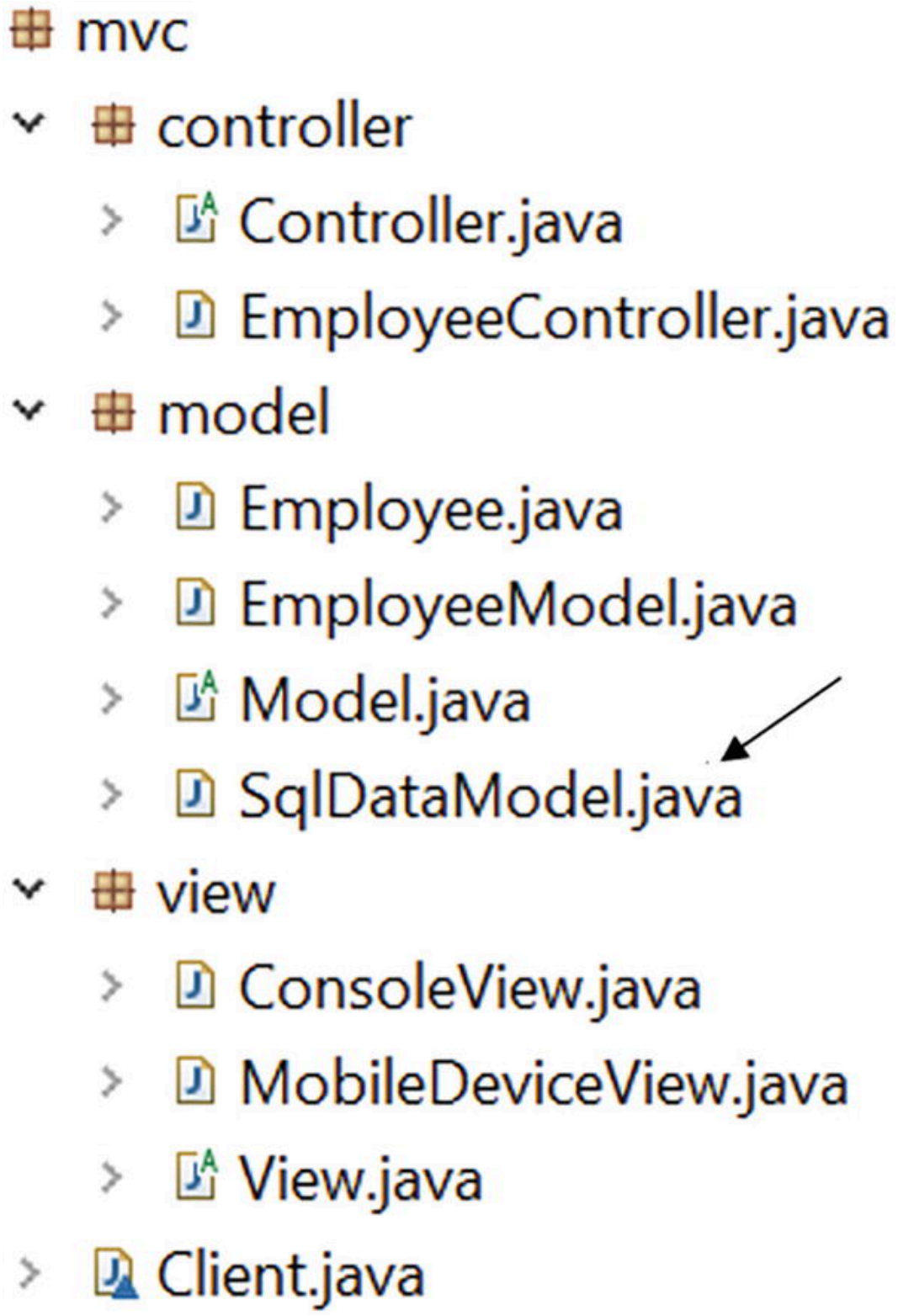

Figure 27-9 New Package Explorer view after the addition of SqlDataModel.java

And then you bind this model with the mobile device view (which was shown previously) as follows:

```
// Testing a new model
model = new SqlDataModel();
view = new MobileDeviceView();
controller = new EmployeeController(model, view);
controller.displayEnrolledEmployees();
```

This segment can produce the following output:

```
 ***This is a mobile view of currently enrolled employees.***
Employee Id      Employee Name

_________________________
E1      Amit
E2      John
E3      Sam
E4      Patrick
++++++++++++++++++++
```

Note that inside `SqlDataModel` you could use a different data structure, such as a `LinkedList` to serve your purpose. This is why you don't declare the `List` (`enrolledEmployees`) in the super class (`Model.java`).

I hope that you can now include a new controller too if you want. In short, you can easily add (or plug in) a model, view, or controller in this system. It is possible because each component is independent of the other and they do not have any direct knowledge about the other components.

I repeat, it is not mandatory to separate components in a similar manner. For example, if you believe that for your application you should combine the view layer and the controller layer, it's ok. To follow the core theory, I separated the layers in these examples. In this context, I like to include the following quote from Trygve Reenskaug (**https://folk.universitetetioslo.no/trygver/themes/mvc/mvc-index.html**):

> *An important aspect of the original MVC was that its Controller was responsible for creating and coordinating its subordinate views. Also, in my later MVC implementations, a view accepts and handles user input relevant to itself. The Controller accepts and handles input relevant to the Controller/View assembly as a whole, now called the Tool.*

This is the end of this part. I hope you enjoyed learning all of the patterns in this book. It is time to look into some other aspects of design patterns, which you'll see in the following part.

‹ 26. Null Object Pattern | Part IV. The Final Talks on Design Patterns ›